AdaRule: LLM-Driven Natural Language to LTL Conversion via Pattern-Adaptive Rule Induction

Jiayi Hu[1], Jingling Sun[2], Chong Wang[3], Yihao Huang[1†], Jincao Feng[4], Yilongfei Xu[1], Yong Li[5], Kailong Wang[6], Weikai Miao[1,4], Jin Song Dong[7], Geguang Pu[1,4]

[1]East China Normal University, China [2]University of Electronic Science and Technology of China, China
[3]Nanyang Technological University, Singapore [4]Shanghai Kong'an Intelligent Technology Software Co., Ltd., China
[5]Key Lab. of System Software (Chinese Academy of Sciences), ISCAS, China
[6]Huazhong University of Science and Technology, China [7]National University of Singapore, Singapore

Abstract

Translating natural language (NL) specifications into Linear Temporal Logic (LTL) formulas is critical for bridging human intent and formal system verification. While large language models (LLMs) have made this task more feasible, adapting NL2LTL systems to different domains remains challenging due to varied linguistic conventions. Existing methods typically rely on manually crafted translation rules or pattern-specific templates, which are costly to construct and do not generalize across domains. A core challenge is that NL–LTL datasets provide paired examples but do not explicitly indicate which linguistic elements reflect pattern-specific translation conventions. To address this problem, we propose AdaRule, a feedback-guided approach for pattern-adaptive NL2LTL translation via automatic rule induction. The key insight is that even without explicit supervision, we can leverage LLMs to perform NL2LTL translation using only basic, general-purpose rules. By comparing the predicted LTL with ground truth LTL formulas, the LLM can identify where the general rules fall short and uses this feedback to induce new translation rules that capture pattern-specific conventions. AdaRule consists of two collaborative LLM-based components: a Translator that performs the NL2LTL conversion, and a Learner that analyzes failed translations to generate corrective rules. These rules are incorporated into the Translator's prompts to improve future predictions. Through iterative learning, AdaRule progressively adapts to pattern-specific conventions without requiring manual engineering. Experiments on four benchmark NL2LTL datasets and three different foundation LLMs show that AdaRule outperforms the other baselines by at least 21.1% on average, demonstrating the effectiveness of automatic rule induction. Our code is available at https://github.com/TrustIntelligence/ADARULE.

CCS Concepts

• **Software and its engineering**; • **Computing methodologies** → **Artificial intelligence**; • **Security and privacy** → **Formal methods and theory of security**;

Keywords

NL2LTL, Large Language Model, Rule Induction, Feedback-guided

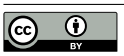
ACM Reference Format:
Jiayi Hu[1], Jingling Sun[2], Chong Wang[3], Yihao Huang[1†], Jincao Feng[4], Yilongfei Xu[1], Yong Li[5], Kailong Wang[6], Weikai Miao[1,4], Jin Song Dong[7], Geguang Pu[1,4]. 2026. AdaRule: LLM-Driven Natural Language to LTL Conversion via Pattern-Adaptive Rule Induction. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3787821

1 Introduction

Formal specification [12] plays a critical role in ensuring the correctness and reliability of systems across domains such as software engineering [45, 54], embedded systems [25, 48], and model verification [12, 28]. Among formal languages, Linear Temporal Logic (LTL) [44] is widely adopted for expressing temporal properties due to its expressive power and compatibility with model checking. However, authoring LTL formulas manually remains a non-trivial task that demands substantial domain expertise and incurs high annotation costs [22]. These challenges continue to hinder the practical adoption of formal methods in real-world development.

To alleviate this burden, some works [7, 41] have explored automatically translating natural language specifications into LTL formulas, a task known as NL2LTL. Although these approaches offer a promising path toward lowering the barrier to formal specification, they are often limited by the linguistic variability of natural language across domains. In practice, users may express the same temporal requirement using widely different linguistic patterns such as surface forms, syntactic structures, or terminology, depending on the domain context [3]. We refer to such dataset-specific linguistic regularities as "patterns". While these linguistic patterns often arise within specific application domains such as banking or manufacturing, our focus is on adapting to the patterns themselves. To address this variation, existing approaches typically rely on manually constructed translation rules tailored to different patterns [13, 21, 37, 38, 56]. While effective within their target contexts, the key challenge lies not in applying these rules but in discovering them, which is labor-intensive and difficult to scale. A promising direction to address this limitation is to automatically induce pattern-specific rules. In particular, LLMs have shown strong potential in this area as they can effectively bridge diverse natural language expressions with consistent logical forms [8]. This makes them a promising tool for learning pattern-specific translation rules in a scalable and adaptable manner.

† Yihao Huang is the corresponding author.



ICSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2025-3/2026/04
https://doi.org/10.1145/3744916.3787821

However, the greater challenge lies in inducing these rules automatically in the absence of explicit supervision. While NL2LTL datasets consist of paired natural language and LTL formulas, they do not explicitly indicate which parts of the input reflect pattern-specific conventions, making it difficult for models to infer the underlying translation patterns. To tackle this challenge, we adopt a feedback-guided contrastive approach that uncovers linguistic patterns through translation failures. We begin by prompting an LLM to perform an NL2LTL task using only a set of basic rules that encode standard logical operators and naming conventions. When the model produces an incorrect translation, its output is compared with the corresponding ground-truth LTL formula. The differences between them reveal precisely where the general rules fall short and indicate the pattern-specific logic that was overlooked. By systematically analyzing these discrepancies, LLMs can distill translation guidelines that reflect the linguistic conventions of the target dataset, without relying on explicit supervision.

With this insight, we propose AdaRule, a pattern-**Ada**ptive **Rule** induction method for NL2LTL translation task that automatically learns guideline rules (*i.e.*, guidelines) for LLMs through iterative feedback. The approach consists of two collaborative LLM-based components. The LLM Translator performs the NL2LTL conversion, while the LLM Learner analyzes failed cases and induces new guidelines. Starting from general prompts, the Translator processes training samples. When a failure occurs, the Learner examines the error and summarizes a guideline specific to the linguistic pattern. This guideline is then incorporated into the Translator's prompt in the next iteration. Through this feedback loop, the system progressively adapts to the dataset-specific linguistic patterns by internalizing linguistic conventions and specification patterns in the form of guidelines. Experiment conducted on four datasets compared with five baselines shows that our method AdaRule outperforms the best baseline by 21.1% on average. We also evaluate the generality of our method across three foundation LLMs, observing consistently high accuracy with averages reaching up to 78% on average, demonstrating strong cross-model adaptability.

In summary, our key contributions are:

- We address the lack of explicit supervision in pattern-adaptive NL2LTL by leveraging differences between LLM-generated and ground-truth LTL formulas as implicit signals to induce pattern-specific guidelines without manual annotations.
- We propose a feedback-guided, automated LLM-based framework. It consists of an LLM Translator for NL2LTL conversion and an LLM Learner that analyzes failures to iteratively refine translation prompts with pattern-specific rules.
- We evaluate our method across four NL2LTL datasets and three foundation LLMs. Results show superior accuracy over existing approaches and strong cross-model generalization.

2 Background and Motivation

2.1 LTL Conversion Task

Linear temporal logic and basic translation rules. LTL [44] is an extension of propositional logic with temporal operators. Its syntax is defined as follows:

$$\varphi ::= p \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \mathbf{G}\varphi \mid \mathbf{F}\varphi \mid \mathbf{X}\varphi \mid \varphi\mathbf{U}\varphi \mid \varphi\mathbf{R}\varphi$$

where $p \in AP$ is an atomic proposition from a finite set of atomic propositions AP. The temporal operators are interpreted over an infinite sequence of states as follows:

- $\mathbf{G}\varphi$ (Globally): φ holds in all future states.
- $\mathbf{F}\varphi$ (Finally): φ eventually holds in some future state.
- $\mathbf{X}\varphi$ (Next): φ holds in the next state.
- $\varphi\mathbf{U}\psi$ (Until): φ holds continuously until ψ becomes true in some future state.
- $\varphi\mathbf{R}\psi$ (Release): ψ must remain true until both φ and ψ are true. Once φ becomes true, ψ is no longer required to be true.

The construction rules for LTL formulas are:

- Any atomic proposition p is an LTL formula.
- If φ and ψ are LTL formulas, then the Boolean formulas $\neg\varphi$, $\varphi \wedge \psi$, $\varphi \vee \psi$, and $\varphi \rightarrow \psi$ are also LTL formulas.
- If φ and ψ are LTL formulas, then $\mathbf{X}\varphi$ (Next), $\varphi\mathbf{U}\psi$ (Until), $\mathbf{G}\varphi$ (Globally), $\mathbf{F}\varphi$ (Finally), and $\varphi\mathbf{R}\psi$ (Release) are LTL formulas.

These rules provide a basic mapping between linguistic constructs and LTL operators and are used as the foundational knowledge.

NL2LTL task. Given a natural language specification NL, the NL2LTL task aims to translate it into an equivalent LTL formula LTL. Formally, it is defined as the following mapping function.

$$\mathcal{T} : NL \rightarrow LTL,$$

such that the generated formula $LTL = \mathcal{T}(NL)$ is semantically equivalent to the input natural language description, accurately capturing its temporal constraints or system behaviors.

2.2 Challenge and High-level Idea

Recent advances in LLMs have improved NL2LTL translation in general settings, but adapting to new linguistic patterns remains challenging. Pattern-specific specifications often contain unique linguistic patterns that general rules miss, and manually creating rules for each dataset is time-consuming and unscalable. Ideally, models would learn pattern-specific rules directly from data. However, existing datasets only offer NL–LTL pairs without indicating which input parts map to pattern-specific logic, making it hard for models to identify translation patterns without explicit supervision.

Our key insight is that this missing supervision can be recovered by contrasting the model's behavior under basic translation rules with the ground-truth output. For example, given the input:

> *"Eventually, the robot must deliver the package after it reaches the charging station."*

An LLM guided only by basic rules will produce:

$$\mathbf{F}\,(\texttt{reach\_charging} \wedge \mathbf{F}\,\texttt{deliver\_package}).$$

However, the correct LTL should be:

$$\mathbf{F}\,(\texttt{reach\_charging} \wedge \mathbf{XF}\,\texttt{deliver\_package}).$$

The discrepancy between these two outputs provides an extra supervision signal, revealing that the word "after" implies a nested temporal structure indicating "next time", which is more accurately captured by the logical operator **XF** rather than a simple **F**. By learning from such differences, the LLM can infer translation patterns that generalize to similar expressions in future NL2LTL tasks. This approach allows the system to adapt to diverse linguistic patterns without relying on manually crafted rules.

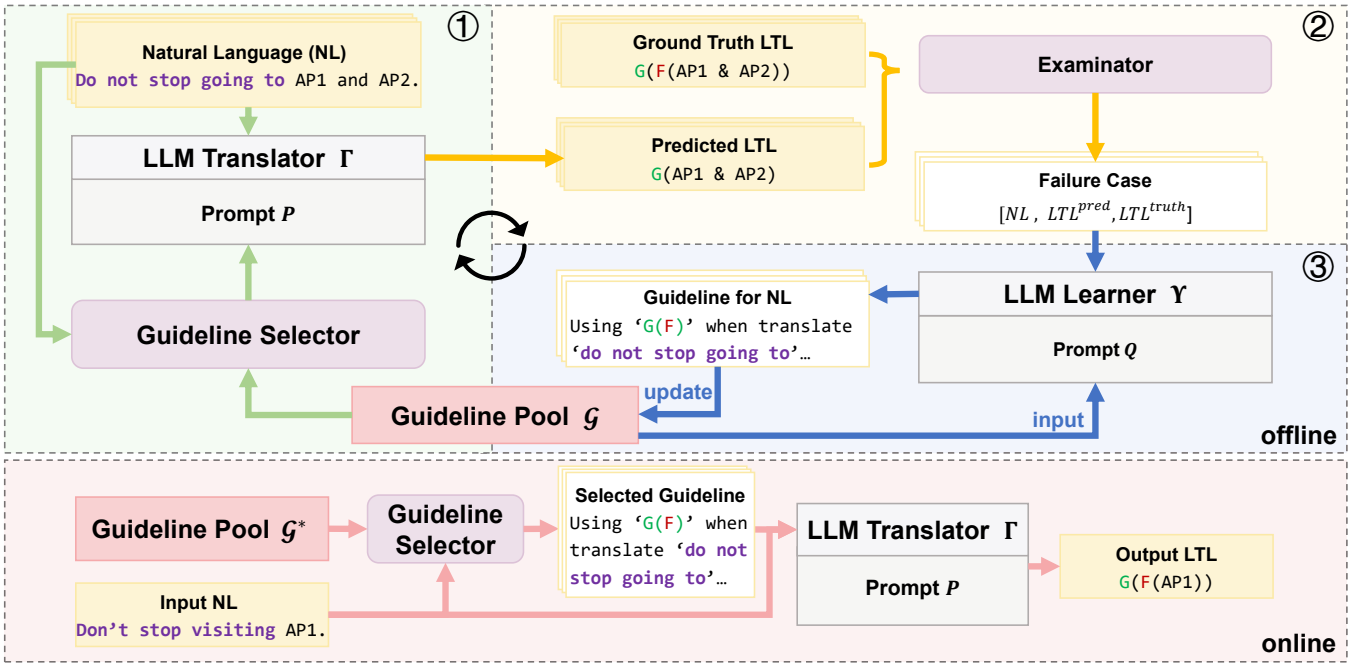


Figure 1: Overview of AdaRule's workflow.

Prompt *P*

```
Task:
Translate NLP Text into LTL formulas.

Basic Rules:
Use the following operators: G, F, X, U, R, !, &, |, ->
The meaning of each operator:
'G' means "globally": G a indicates that a is true in all
future states.
……

Guidelines:
<Guidelines>

Output format:
Please respond in the following format:
The final LTL translation is: [LTL formula].FINISH

The source natural language is: <NL>
```

Figure 2: Prompt P for LLM Translator Γ.

Prompt *Q*

```
Task:
Learn the rule characteristics of the NL to LTL task based
on the information provided, and update the Guidelines.

Requirement:
The new rules should: address the problems exposed by
Translator in the failure cases, help Translator understand
the mapping relationship between NL and LTL.

Input information:
The existing guidelines pool: <Guidelines>
The failure case [NL, LTL^pred, LTL^truth] is as follows:
<Failure Case>

Output format:
You should respond strictly in the following format:
<start>"guideline"<end>
```

Figure 3: Prompt Q for LLM Learner Υ.

3 Our Approach

We propose AdaRule, a feedback-guided approach for pattern-adaptive LTL conversion using LLMs. Starting with a general model and base prompt, AdaRule iteratively refines pattern-specific translation rules by learning from conversion failures, enabling automatic adaptation and improved accuracy in the target datasets.

3.1 Overall Workflow

In Figure 1, AdaRule consists of two stages powered by LLMs: an offline learning stage, which iteratively learns a pool of pattern-specific conversion guideline rules (*i.e.*, guidelines) from conversion samples, and an online conversion stage, which translates a given natural language description into LTL using the learned guidelines.

In the offline learning stage, as illustrated in Algorithm 1, AdaRule employs an LLM-based translation procedure, denoted as **Translate**, to convert each NL description in the training set $\mathcal{D}_{\text{train}}$ into an LTL formula using a prompt template shown in Figure 2 and incorporating relevant guidelines selected from the guideline pool $\mathcal{G}$. Initially, $\mathcal{G}$ is empty. After each translation, an **Examine** procedure compares the predicted LTL formula with the ground-truth LTL to check their semantic equivalence. Translation failures, where the predicted LTL formula is not semantically equivalent to the ground-truth LTL, are collected into a failure case set $\mathcal{F}$. After traversing all the training samples, each failure case is passed to an LLM with the prompt template shown in Figure 3 to induce the corresponding new guideline g. The newly learned guideline g is then updated to

the guideline pool $\mathcal{G}$ to improve the pattern-specific translation effectiveness of Translate, such as enhancing its ability to adapt terminology specific to the target linguistic patterns in subsequent iterations. This guideline induction process is referred to as **Learn**. The Translate, Examine, and Learn procedures iterate until a maximum iteration limit I is reached, yielding the final guideline pool $\mathcal{G}^*$. Note that there is a mapping set $\mathcal{A}$ maintained to record the one-to-one correspondence between each NL description in the training set $\mathcal{D}_{\text{train}}$ and its associated guideline.

In online conversion stage, given an NL description in the test dataset, AdaRule translates it into an LTL formula using the Translate procedure empowered by the final guideline pool $\mathcal{G}^*$ learned during the previous offline stage.

3.2 LLM-based LTL Translation

We use an **LLM Translator** $\mathbf{\Gamma}$ to perform the **Translate** procedure. The LLM Translator takes a prompt template P that describes the task and rules as input. As the core generation component in our approach, $\mathbf{\Gamma}$ leverages the semantic understanding and reasoning capabilities of LLMs.

Overall process. The prompt template P includes (1) a task description, (2) a set of basic rules, (3) detailed instructions for the expected output format, (4) a <Guidelines> placeholder (as shown in red text of Figure 2) for selected guidelines, and (5) a <NL> placeholder for the input natural language NL. The basic rules define the naming conventions for atomic propositions and explain the semantics of standard LTL operators (*e.g.*, **G** denotes "globally", so **G** a means that a holds in all future states).

For a given natural language input NL and a guideline pool $\mathcal{G}$, the specific prompt p is constructed by augmenting P with NL and a selected subset of NL-related guidelines $\mathcal{G}_{NL}$ retrieved from $\mathcal{G}$ with a guideline selector. Specifically, the prompt is defined as:

$$p = P \oplus NL \oplus \mathcal{G}_{NL}, \tag{1}$$

where $\oplus$ denotes a text filling operation, inserting NL and $\mathcal{G}_{NL}$ into the designated <NL> and <Guidelines> placeholders within P respectively. The LLM Translator generates the corresponding LTL formula for NL as:

$$LTL^{\text{pred}} = \mathbf{\Gamma}(p). \tag{2}$$

This process defines the LTL generation for the input NL.

Guideline selector. To enhance the effectiveness of our LTL translation procedure, it is essential that the LLM Translator focuses on the most relevant guidelines for each specific input. Directly including all available guidelines in the prompt not only increases the computational cost but may also dilute useful information due to the limited attention span of LLMs. Therefore, we introduce a *selection strategy* to support targeted prompt construction, enabling the model to condition on a subset of informative and semantically relevant guidelines from the full guideline pool $\mathcal{G}$.

To implement this strategy, we compute the semantic similarity between the current input NL and each training example to select the most relevant guideline. This is enabled by a one-to-one mapping between training samples and guidelines. Specifically, given a training set of size M, we construct a guideline pool of the same size, where each training sample $NL_m \in \mathcal{D}_{\text{train}}$ is associated with a corresponding guideline g_m, forming the mapping $\mathcal{A} = \{(NL_m, g_m)\}$.

Algorithm 1 Iterative LTL Conversion and Guideline Induction

Require: Training Set $\mathcal{D}_{\text{train}}$, Maximum Iteration Limit I
Ensure: Final Guideline Pool $\mathcal{G}^*$

$\mathcal{G} \leftarrow \emptyset$
$\mathcal{A} \leftarrow \{\}$
for $i = 0$ to I **do**
  $\mathcal{F} \leftarrow \emptyset$
  for $(NL, LTL^{\text{truth}}) \in \mathcal{D}_{\text{train}}$ **do**
    $LTL^{\text{pred}} \leftarrow$ **Translate**$(NL, \mathcal{G})$
    if **Examine**$(LTL^{\text{pred}}, LTL^{\text{truth}})$ different **then**
      $\mathcal{F}.push(< NL, LTL^{\text{pred}}, LTL^{\text{truth}} >)$
    end if
  end for
  for $f \in \mathcal{F}$ **do**
    $g \leftarrow$ **Learn**$(f, \mathcal{G})$
    $\mathcal{A}[f.NL] \leftarrow g$
    $\mathcal{G}.update(g)$
  end for
end for
return $\mathcal{G}$

In cases where a training sample does not yield a guideline, the corresponding g_m is left empty. This mapping allows each guideline to be traced back to its source sample, enabling relevance assessment between the input NL and guidelines based on the similarity between NL and NL_m.

We use a sentence embedding model (*e.g.*, `all-MiniLM-L6-v2` [1]) to encode both NL and all NL_m into vector representations. Semantic similarity is measured using cosine similarity:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}, \tag{3}$$

where $\mathbf{A}$ and $\mathbf{B}$ denote the embeddings of NL and NL_m, respectively.

Based on these similarity scores, we retrieve the top-K most relevant guidelines and insert them into the Translator's prompt to support pattern-adaptive conversion.

3.3 Automata-based Equivalence Examination

For the **Examine** procedure, a predicted LTL formula LTL^{pred} is evaluated with its corresponding ground-truth formula LTL^{truth}. Since different syntactic forms may express the same logical semantics, we adopt an automata-based equivalence examiner rather than a simple string match.

Specifically, we use the Spot library [15], which converts LTL formulas into Büchi automata and checks for mutual implication:

$$\text{Equiv}(LTL_1, LTL_2) = (LTL_1 \rightarrow LTL_2) \wedge (LTL_2 \rightarrow LTL_1). \tag{4}$$

If the two formulas are not semantically equivalent, or if parsing fails due to syntactic or structural errors, the corresponding case is marked as a failure. Each failure is represented as a tuple:

$$(NL, LTL^{\text{pred}}, LTL^{\text{truth}}), \tag{5}$$

where NL is the natural language input, and the two LTL formulas represent the prediction and ground truth, respectively.

[1] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

All such failure tuples in the training iteration are aggregated into a failure set $\mathcal{F}$, which is then used as the input for our guideline learning procedure in subsequent steps.

3.4 LLM-based Guideline Learning

LLM Learner Υ is our key component that performs the **Learn** procedure, where new guidelines are induced from translation failures. Its goal is to analyze the failure cases when the LLM Translator produces incorrect LTL formulas and to generate corrective guidelines that help avoid similar errors in future translations. For each failure case, the LLM Learner generates one corresponding guideline to ensure specificity and relevance. The induced guidelines are updated to the guideline pool and become available to the LLM Translator in the next iteration. This incremental feedback-guided process allows the system to adapt to underlying pattern-specific language conventions without requiring any manually labeled translation guidelines.

The LLM Learner takes a general prompt template Q that defines the task and the desired guideline format as input. The prompt template Q includes (1) a <failure_case> placeholder for a failure case f in the failure case set $\mathcal{F}$, and (2) a <Guidelines> placeholder for the current guideline pool $\mathcal{G}$. Formally, the prompt q input to the LLM Learner is constructed as:

$$q = Q \oplus f \oplus \mathcal{G}, \tag{6}$$

where $\oplus$ denotes a text filling operation that injects content into corresponding designated placeholders within the prompt. The guideline induction step is defined as:

$$g = \Upsilon(q). \tag{7}$$

This process defines the guideline learning for each iteration.

To facilitate traceability and guideline reuse, we maintain a one-to-one mapping $\mathcal{A} = (NL_m, g_m)$ between each natural language sample NL_m and its associated guideline g_m. This mapping is dynamically updated: if a sample continues to produce incorrect translations in subsequent iterations, its associated guideline is replaced by a newly induced one. A g_m induced for a sample NL_m is not guaranteed to be correct at the time of induction. Its usefulness is assessed in future iterations, when it is applied again to NL_m and the predicted LTL is compared with the ground truth. If the prediction remains incorrect, NL_m stays in the failure set and is used to generate a new guideline to replace g_m. This mechanism ensures that the guideline pool $\mathcal{G}$ remains continuously updated with guidelines that are refined through repeated application.

4 Evaluation

Our evaluation aims to answer these research questions:

- **RQ1**: How effective is our approach in the translation of NL2LTL on benchmarks from different linguistic patterns?
- **RQ2**: How does our approach perform when applied with different foundation LLMs?
- **RQ3**: What is the contribution of LLM Learner in our approach to its accuracy?
- **RQ4**: How do different strategy and parameter choices within our approach affect its accuracy?

4.1 Evaluation Setup

Datasets. To comprehensively evaluate AdaRule 's generalization ability, we select four publicly available NL–LTL datasets. As several source papers do not provide official dataset names, we consistently use the shorthands "SynthNL" [14], "SpecNL" [13], "ConfoNL" [51], and "LangNL" [37] for clarity. The datasets are chosen based on a systematic review of recent NL2LTL survey [4], complemented by a supplementary keyword search (*e.g.*, "NL2LTL", "LTL translation") in major academic databases (*e.g.*, arXiv, IEEE Xplore) covering 2024–2025 publications, and by a case-by-case analysis to exclude unsuitable datasets (*e.g.*, those intended for interactive synthesis or producing non-LTL outputs). This process ensures coverage of diverse linguistic patterns. **SynthNL** [14, 56] contains 240k training and 15k test NL–LTL pairs derived from eight specification patterns across multiple application scenarios (*e.g.*, aviation, computer operations). We sample 500 examples (400 train/100 test) due to resource limits. **SpecNL** [13] comprises 36 difficult formal specifications with special linguistic constructs and serves as a standard benchmark for translation precision [13, 56]. We split the examples into 18 for training and 18 for testing. **ConfoNL** [51] is derived from real-world robotic manipulation tasks and captures sequential, action-oriented instructions. After preprocessing [30], we obtain fewer than 500 unique samples. We randomly select 300 examples (200 train/100 test) for experiments. **LangNL** [37] focuses on semantically diverse commands for robotic navigation, exhibiting greater structural and linguistic complexity. We randomly sample 500 examples (400 train/100 test) from the training split.

To facilitate guideline learning and application, we anonymize atomic propositions (APs) as placeholders AP_i [30]. This abstraction reduces noise from task-specific naming, letting the model focus on structural patterns over lexical variation, improving generalizability and applicability across diverse linguistic patterns.

Metric. We adopt *translation accuracy* (Accuracy) as our primary evaluation metric. We use Spot [2] [16], a widely used LTL model-checking library, to verify equivalence between LTL formulas generated by AdaRule and the ground-truth LTL. The final result is the accuracy on the test set when using the guideline set $\mathcal{G}^*$ obtained at the final iteration, which is defined as:

$$\text{Acc} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{NL_n \in \mathcal{D}_{\text{test}}} 1[\text{Equiv}(LTL_n^{\text{pred}}, LTL_n^{\text{truth}}) = \text{True}],$$

where $\mathcal{D}_{\text{test}}$ is the test set, and $LTL_n^{\text{pred}} = \Gamma(P \oplus NL_n \oplus \mathcal{G}_n^*)$ is the LTL formula generated for NL_n under guideline pool $\mathcal{G}^*$.

Baselines. We compare our method with four state-of-the-art NL2LTL methods. **NL2SPEC** [13] is an interactive approach that decomposes NL inputs into sub-translations, allowing users to iteratively correct them. **LfF** [56] improves NL2LTL translation by enabling users to correct sub-translations during interaction. **NL2LTL** [21] is a rule-based system that maps NL inputs to LTL formulas using predefined `DECLARE` templates selected via LLMs. **CoT-TL** [38] integrates Chain-of-Thought reasoning and Semantic Role Labeling to generate candidate LTL formulas. We follow each method's official implementation and keep rules and LLM settings the same as ours to ensure comparability.

[2] https://spot.lre.epita.fr/

Table 1: Compare with baselines

Method	SynthNL	SpecNL	ConfoNL	LangNL	AVG
FLAN-T5 [11]	56.0%	0	3.0%	40.0%	24.75%
NL2LTL [21]	0	0	1.0%	0	0.25%
NL2SPEC [13]	42.0%	66.7%	56.0%	27.0%	47.93%
LfF [56]	89.0%	77.8%	49.0%	26.0%	60.45%
CoT-TL [38]	20.0%	61.1%	42.0%	45.0%	42.03%
AdaRule	**92.0%**	**83.3%**	**83.0%**	**68.0%**	**81.58%**

In addition to existing NL2LTL methods, fine-tuning a language model is a standard baseline, with **FLAN-T5** [11] being widely adopted in recent work [10, 36, 37, 56]. Accordingly, we fine-tune the FLAN-T5 variant on each training dataset to perform NL2LTL.

Implementation details. In our method, the base LLM is DeepSeek-V3. The number of training iterations is set to 5. In the translation component, we select guidelines based on cosine similarity with training-set-aligned retrieval. Each translation prompt includes 16 guidelines, all extracted from training-set translation errors. In the rule induction component, one sample is processed per learning step, and one guideline is generated per learning step. We set the temperature of all LLMs to 0 for reproducibility, except Gemini, which occasionally outputs malformed characters. All experiments are conducted on an Ubuntu 20.04 server equipped with dual 16-core CPUs, using Python 3.10.16.

4.2 RQ1: Effectiveness of our method

To evaluate the effectiveness of our method in NL2LTL translation, we conduct a comparison with five representative baseline methods (NL2SPEC, LfF, NL2LTL, CoT-TL, FLAN-T5) across four different datasets (SynthNL, SpecNL, ConfoNL, LangNL) in Table 1.

Comparison with baselines. As shown in Table 1, our method consistently achieves the best performance across all four datasets (SynthNL, SpecNL, ConfoNL, LangNL), with an average accuracy (AVG) of 81.58%, significantly surpassing all baselines. Compared to the best baseline in others, LfF, which achieves an average accuracy of 60.45%, our method shows an improvement of over 21%. The advantage is even more substantial when compared with other baselines such as NL2LTL (0.25%), NL2SPEC (47.93%), CoT-TL (42.03%), and FLAN-T5 (24.75%). These results demonstrate the effectiveness of our approach for the NL2LTL task. Furthermore, the consistently high accuracy across diverse datasets highlights the strong generalizability of our method across varying datasets.

Overfit issue. To avoid overfitting from large datasets, we verify that our method works effectively without large-scale training. On SynthNL, we apply guidelines induced from 400 training samples to the full 15k test set, achieving 93.2% accuracy, comparable to 92.0% on the 100-sample test set. This shows our method performs well without large-scale training and has a low risk of overfitting.

Correctness of Spot. While Spot is the de facto standard tool in the LTL community, no tool guarantees 100% accuracy. To validate the reliability of our evaluation, two logic experts manually verify all four test sets (318 samples), with results fully consistent with Spot. We further validate this using an independent LTL satisfiability checking tool, Aalta [33]. The results are identical to those of Spot, confirming the correctness of our semantic equivalence check.

Correct case study. Here, we demonstrate that our method can successfully capture implicit linguistic patterns, even when no explicit NL description directly matches the formal logic structure.

Example
NL: if either a AP1 or a AP2 then at some point in time the AP2 and the AP3.
Ground-truth LTL: G((AP1 | AP2) → **F**(AP2 & AP3))
Used Guideline: For statements involving "at some point in time" followed by sequential conditions, ensure the implication structure (condition → **F**(consequence)) is globally enforced (**G**) to correctly represent the persistent conditional relationship, rather than treating the conditions as independent eventualities combined with logical AND (&).

This example is taken from SynthNL, a dataset simulating high-assurance domains such as autonomous systems, where safety properties must always hold. The natural language phrase "at some point in time" maps to the temporal operator **F**. The overall implication is wrapped in a global operator **G**, reflecting the requirement that the condition must hold at all times. Our method successfully generates the correct LTL formula by leveraging learned guidance from similar global patterns in the training data.

Error case study. Although our method significantly outperforms other baselines and achieves high accuracy, it is not entirely error-free in the test dataset. To better understand the remaining failure cases, we conducted a detailed analysis and found that most of them are not caused by flaws in our method itself. Instead, the errors can be broadly categorized into two types: (1) issues with the test dataset (Case 1), and (2) limitations in the LLM's ability (Case 2).

Case 1
NL: Either AP1 holds infinitely often or AP2 holds in the next step.
Predicted LTL: (**GF**AP1) | (**X**AP2)
Ground-truth LTL: **G**(**F**(AP1) | **X**(AP2))

Case 1. This case is from the dataset SpecNL, predicted by DeepSeek-V3. Semantically, the phrase "AP1 holds infinitely often" is typically interpreted as AP1 occurring repeatedly in the future, which is more accurately expressed in LTL as **GF**AP1. However, the ground-truth formula adopts **G**(**F** AP1...), meaning "in every state, AP1 will eventually occur". This indicates a rare labeling error in the test set, and is not the error of our method.

Case 2
NL: first go to AP2 once then go to AP1 once while avoiding AP3 then go to AP3 once while avoiding AP4 finally go to AP4.
Predicted LTL: (!AP1 **U** AP2) & **G**!AP3 & (!AP3 **U** AP1) & **G**!AP4 & (!AP4 **U** AP3) & **F**(AP4)
Ground-truth LTL: (!(AP1) **U** AP2) & ((!(AP3) **U** AP1) & ((!(AP4) **U** AP3) & **F**(AP4))) & ((!(AP2) **U** (AP2 **U** (!(AP2) **U** AP1))) & ((!(AP1) **U** (AP1 **U** (!(AP1) **U** AP3))) & (!(AP3) **U** (AP3 **U** (!(AP3) **U** AP4)))))

Case 2. This case is from the dataset LangNL, where each AP denotes a target destination in the navigation task. The mapping between the natural language description and the corresponding LTL formula is more complex in this case. The LTL formula predicted by the LLM (DeepSeek-V3) captures only part of the key temporal relations, failing to fully represent all constraints and nested

Table 2: Effect with different foundation LLMs

LLM	SynthNL	SpecNL	ConfoNL	LangNL
DeepSeek	92.0%	83.3%	83.0%	68.0%
Qwen	85.0%	66.7%	69.0%	48.0%
Gemini	95.0%	72.2%	78.0%	91.0%

Table 3: Assessing the contribution of LLM Learner

Method	SynthNL	SpecNL	ConfoNL	LangNL
Expert-crafted guideline pool	59.0%	77.8%	71.0%	41.0%
One-shot learned guideline pool	52.0%	77.8%	70.0%	56.0%
AdaRule	**92.0%**	**83.3%**	**83.0%**	**68.0%**

structures. Although the guideline pool contains guidelines covering patterns such as "avoiding" and "once", deriving an accurate formal specification remains challenging. This indicates limitations in DeepSeek's understanding capabilities. In contrast, our method based on Gemini-2.5-Flash can successfully generate the correct LTL formula in this case. Such failures highlight that, in complex scenarios, the ability of LLM plays a crucial role in generating LTL.

We further analyze the induced guidelines and find them high-quality. For example, the model learns to generalize double-negative expressions (*e.g.*, "prohibited from not") into positive temporal requirements before LTL translation. The induced guidelines may be general (covering multiple linguistic variants with a single rule) or case-specific, as long as they effectively guide NL2LTL translation.

Discussion of Findings for RQ1. The results in Table 1 demonstrate the strong effectiveness of AdaRule, with an average accuracy improvement of 21.1% over the best baseline. This improvement is largely attributable to our method's ability to derive more concise guidelines from failure cases, better capturing the NL2LTL mapping patterns. A limitation is the lower performance on LangNL, mainly due to its complex ground-truth LTL formulas. Specifically, the average length of formulas and number of temporal operators are: SynthNL (26.20, 5.75), SpecNL (19.22, 4.72), ConfoNL (11.03, 3.31), and LangNL (57.05, 12.58), indicating that LangNL formulas are significantly longer and more complex.

4.3 RQ2: Generality of our method

Since our method depends on LLMs to perform the NL2LTL task, it is important to evaluate its generalization across different foundation LLMs. We conduct experiments using three representative LLMs: DeepSeek-V3 [3], Qwen2.5-72B-Instruct [4], and Gemini-2.5-Flash [5]. We use the four datasets: SynthNL, SpecNL, ConfoNL, and LangNL. In the evaluation, we keep all other components fixed and only vary the relevant LLMs to isolate their impact.

The results are presented in Table 2. Overall, our method exhibits strong and consistent performance across all four datasets and LLM variants, highlighting its adaptability to different LLM capabilities. For example, on the SynthNL dataset, it achieves 95.0% accuracy with Gemini-2.5-Flash, 92.0% with DeepSeek-V3, and 85.0% with Qwen2.5-72B-Instruct. DeepSeek-V3 leads on SpecNL and ConfoNL, reaching 83.3% and 83.0% on these two datasets, followed by Gemini with 72.2% and 78.0%, respectively. On LangNL, Gemini shows a clear advantage with 91.0% accuracy, outperforming DeepSeek (68.0%) and Qwen (48.0%).

We also observe that different LLMs may reach their best performance at different stages of the iterative training process. For example, on ConfoNL, Gemini-2.5-Flash achieves 84.0% accuracy at the second iteration, surpassing its final score of 78.0% at the fifth iteration. Similarly, Qwen2.5-72B-Instruct peaks at 77.8% on SpecNL during the third iteration. These observations indicate that model-specific behaviors should be taken into account when selecting the number of feedback iterations to achieve optimal performance.

Discussion of Findings for RQ2. The results in Table 2 demonstrate that AdaRule performs consistently across different LLM backbones. Its generality relies on three essential capabilities: understanding natural language instructions [6], reasoning over LTL [13], and inducing abstract rules from examples [58]. LLMs lacking these capabilities, such as Llama2-7B, perform poorly, achieving 0% accuracy, whereas stronger models like Gemini achieve the highest average performance, about 3% higher than DeepSeek. This trend aligns with external benchmarks [1, 49], where Gemini-2.5-Flash consistently outperforms DeepSeek-V3 and Qwen2.5-72B-Instruct on reasoning-intensive tasks. These results underscore that model capability is crucial to our framework's effectiveness, and future LLMs will likely enhance it further.

4.4 RQ3: Effectiveness of LLM Learner

As a key component of our method, the LLM Learner Υ plays a crucial role in overall performance. To evaluate its contribution, we conduct experiments on four datasets: SynthNL, SpecNL, ConfoNL, and LangNL, using different guideline induction strategies.

Assessing the contribution of LLM Learner. We compare our method with two other strategies defined as follows:

- **Expert-crafted guideline pool**: experts extract 16 representative guidelines from the training data, which are combined with the initial prompt to form the expert-crafted prompt pool used by the LLM Translator for NL2LTL translation.
- **One-shot learned guideline pool**: guidelines are induced by the LLM Learner in a single pass over the training data without iteration or feedback. They are then combined with the initial prompt to form a one-shot prompt used by the LLM Translator.

The experimental results, shown in Table 3, demonstrate a clear performance progression. The expert-crafted guideline pool, derived from analyzing the training data, leads to substantial improvements in translation accuracy. On SynthNL and ConfoNL, for example, accuracy increases to 59.0% and 71.0% respectively, demonstrating the benefit of incorporating pattern-specific expertise. The one-shot learned guideline pool achieves similar performance to the expert-crafted strategy across all datasets. It achieves 56.0% accuracy on LangNL and matches expert-level performance on SpecNL (77.8%) and ConfoNL (70.0%), even without iterative refinement. The result underscores the effectiveness of automatically generated guidelines. Our method achieves the highest accuracy on all datasets, including 92.0% on SynthNL and 83.0% on ConfoNL. These results underscore the necessity of incorporating iterative, feedback-guided guideline induction into NL2LTL translation.

[3] https://www.deepseek.com/
[4] https://huggingface.co/Qwen/Qwen2.5-72B-Instruct
[5] https://deepmind.google/models/gemini/flash/

Table 4: Results under different guideline selection strategies

Mode	ENLS	LNLS	LDR	UA1	UA2	RS
Acc (%) ↑	92.0	92.0	89.0	91.0	**95.0**	70.0
Time (min) ↓	**327**	2697	566	354	332	349
Tokens (M) ↓	**7.59**	81.05	40.42	35.44	35.29	11.03

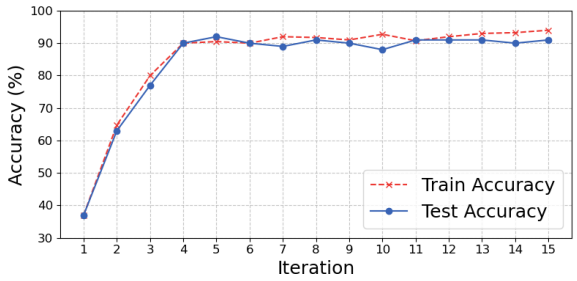


Figure 4: Parameter analysis on the iteration number.

Discussion of Findings for RQ3. The result highlights the effectiveness of AdaRule. One-shot rules achieve an average accuracy of 64.0%, which is 18% lower than our method and up to 40% lower on the SynthNL dataset. This demonstrates that progressively updating and refining guidelines through multiple iterations substantially improves translation accuracy.

4.5 RQ4: Ablation and Sensitivity Analysis

We try to study how different settings and parameters within our method affect its performance. The ablation study on design choice includes the guideline selection strategy. The parameter sensitivity analysis covers: ❶ the number of training iterations, ❷ the number of guidelines used in LLM Translator, ❸ the failure-driven vs. full-data-driven guideline learning, ❹ the number of failure cases used for per guideline generation, and ❺ the number of guidelines generated from per failure case.

For each factor, we conduct experiments on the SynthNL dataset using DeepSeek-V3, varying only the parameter under analysis while keeping all other settings fixed to their default values, as defined in our implementation details. The SynthNL dataset contains lots of samples and covers eight distinct LTL specification patterns, making it well-suited for large-scale learning and evaluation. This setup allows us to isolate the effect of each individual choice.

Ablation study - Guideline selection strategy. The strategy for dynamically selecting guidelines is important in enabling the LLM Translator to focus on guidelines relevant to each NL sample. We evaluate five candidate strategies and select the one that yields the best performance in the experiment.

- **LLM-based direct relevance evaluation (`LDR`):** This strategy uses the LLM to assess relevance of all guidelines to the test sample and selects the most relevant guidelines.
- **Embedding-based NL similarity evaluation (`ENLS`, final solution in our method):** Each guideline is derived from a specific training example. To select relevant guidelines for a test sample, we retrieve the most similar training examples using cosine similarity and reuse their associated guidelines.
- **LLM-based NL similarity evaluation (`LNLS`):** Unlike the `ENLS`, which relies on embedding-based similarity, this strategy leverages an LLM to directly assess the semantic similarity between the test sample and all training samples.
- **Use-all:** This strategy applies all guidelines to each translation task without filtering. To study the impact of sequential ordering, we test two schemes: (1) **UA1**, where guidelines follow their original generation order, and (2) **UA2**, where guidelines are sorted by descending cosine similarity between the test sample and the training sample from which they were generated.
- **Random selection (`RS`):** This strategy selects guidelines from the existing guidelines pool randomly.

Table 4 summarizes the performance of different guideline selection strategies. Among all strategies, UA2 achieves the highest translation accuracy at 95.0%, outperforming the others. The remaining four strategies, including LDR, ENLS, LNLS, and UA1, yield comparable accuracy ranging from 89.0% to 92.0%. These results indicate that providing a sufficient number of highly relevant guidelines is critical for enhancing the performance of the LLM Translator. In addition, all strategies that incorporate relevance-based selection consistently outperform the "RS" strategy, which confirms that the LLM is capable of identifying and using a subset of guidelines that are truly useful for the current input.

In terms of computational efficiency, LNLS incurs the highest cost, with over 2,600 minutes of runtime and 81 million tokens due to repeated LLM-based similarity computations. In contrast, ENLS achieves high accuracy with significantly lower time (327 min) and token usage (7.59M tokens). It presents a more favorable trade-off between performance and cost, offering a better performance-cost trade-off and is adopted as our default.

❶ **Number of training iterations.** In this study, we vary the number of training iterations from 1 to 15 and evaluate the performance of the model at each stage. Specifically, after each iteration, we extract the entire guidelines pool and perform the translation task with it independently on both the training and test sets. This allows us to assess how the model evolves throughout the learning process. Results are shown in Figure 4.

On the training set, accuracy steadily improves as the number of iterations increases. In the early stages, the model rapidly learns effective guidelines, which leads to a significant rise in training accuracy from approximately 40% at iteration 1 to 90% at iteration 5. This reflects the model's growing ability to fit the training data through iterative optimization. On the test set, a similar trend is observed. Accuracy increases from 37% at iteration 1 to 90% at iteration 4, after which the performance stabilizes and fluctuates slightly around 90%.

As test performance stabilizes by iteration 5, we set the number of training iterations to 5 by default. Additional iterations yield minimal performance gains while increasing computational cost. These results also demonstrate that our method can efficiently learn effective guidelines within just a few iterations.

❷ **Number of guidelines used in the LLM Translator.** We investigate how the number of guidelines used in the LTL conversion task affects the performance of our method: 1, 2, 4, 8, 16, and 32.

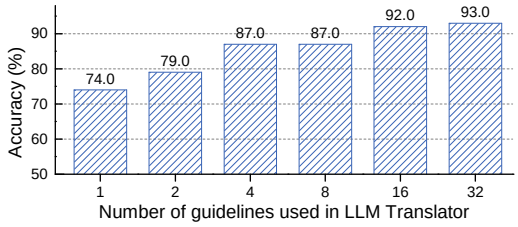


Figure 5: Analyze number of guidelines used in Translator.

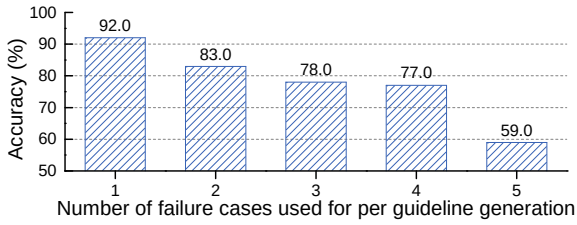


Figure 6: Analysis on the number of failure cases used for per guideline generation.

As shown in Figure 5, accuracy increases consistently as the number of guidelines grows, reaching 92.0% when 16 guidelines are included. Beyond this point, the improvement plateaus, indicating diminishing returns. In contrast, using very few guidelines (*e.g.*, 1 or 2) results in a substantial drop in accuracy, highlighting the importance of providing sufficient and relevant guidance for effective LLM-based conversion.

While increasing the number of guidelines improves performance, it also extends prompt length, raising the computational cost and risking context-related limitations. To balance quality and efficiency, we adopt 16 guidelines per translation in our final configuration, which provides high accuracy with reasonable overhead.

❸ Failure-driven vs. full-data-driven guideline learning. To investigate how the selection of training samples influences the effectiveness of guideline generation, we compare two strategies. The first uses the full data in the training set (full-data-driven), where all NL2LTL conversion samples are included for learning. The second uses only the failure cases from each training iteration (failure-driven), where the model generates wrong LTL formulas.

We find that learning from failure cases yields 92.0% test accuracy, significantly outperforming the 82.0% achieved using all training data. This suggests that failure-driven samples are more effective for guideline induction, as they help correct specific model weaknesses while avoiding redundant or less relevant guidelines. Focusing on these cases also reduces data volume, lowering both runtime and token usage during the induction process.

❹ Number of failure cases used for per guideline generation. To evaluate how the number of failure cases for per guideline generation in LLM Learner affects performance, we examine model accuracy when the guideline induction component generates one rule from a range of 1 to 5 training samples.

As shown in Figure 6, using a single sample per generation yields the highest accuracy (92%). Performance drops noticeably as more samples are used, suggesting that aggregating multiple samples during rule induction leads to overly generalized guidelines. These broad guidelines often fail to capture specific corrective patterns, reducing their precision and effectiveness in NL2LTL translation.

When using more than one sample per guideline generation step in the LLM Learner, two common issues typically arise. Below, we present two representative guidelines induced under such settings to illustrate these issues.

Issue 1. The learned guideline may focus only on a single sample among the multiple provided failure cases. As shown in the guideline in sample 1, among the two input samples, only the second sample (see orange text) is regarded. During the learning process, the model happened to focus on this particular sample while ignoring information from the others. Such a bias toward individual samples can result in guidelines that lack generality and fail to generalize to broader inputs.

Sample 1

failure cases ([$NL, LTL^{pred}, LTL^{truth}$]) and guideline:

- 1. [when a AP1 then as a AP2, sooner or later a AP3 afterwards, **G**AP1→(AP2&**F**(AP3)), **G**((AP1)→**G**((AP2)→**F**(AP3)))]
- 2. [each time the AP1 then if the AP2 then eventually, the AP3, **G**AP1→**G**(AP2→**F**(AP3)), **G**((AP1)→**G**((AP2)→**F**(AP3)))]

Guideline: For nested conditional statements involving "each time...then if...then eventually", ensure the entire implication structure is globally enforced (**G**) and correctly scoped with nested implications (→) and eventualities (**F**), using the pattern **G**(**X** → **G**(Y→**F**(Z))) to represent persistent dependencies and avoid incorrect conjunction placements.

Issue 2. The learned guideline may incorrectly merge different temporal cues from various samples, leading to confused or inappropriate logical structures. As shown in sample 2, the input samples include a mix of temporal expressions such as "after", "every time". During the rule induction process, the model attempts to generalize a single guideline across all samples. As a result, the induced guideline vaguely recommends using "globally operators" for all statements containing "after", "every time". However, "after" emphasizes sequential ordering of events and is not inherently aligned with the semantics of the "**G**" (globally) operator. Misapplying it can lead to incorrect logical interpretations.

Sample 2

failure cases ([$NL, LTL^{pred}, LTL^{truth}$]) and guideline:

- 1. [finally, both the AP3 and the AP4 after either the AP1 or the AP2, **F**(((AP1|AP2)&**F**(AP3&AP4))), **G**((AP1|AP2)→**F**(AP3&AP4))]
- 2. [finally, either AP1 or AP2 and, it is going to happen that AP3 afterwards, **F**((AP1|AP2)&**F**(AP3)), **G**(!(AP1|AP2))|**F**((AP1|AP2)&**F**(AP3))]
- 3. [every time the AP1 and the AP2 then it will happen that the AP2 and the AP3, **G**((AP1&AP2)→(AP2&AP3)), **G**((AP1&AP2)→**F**(AP2&AP3))]

Guideline: For statements involving temporal sequences like "after", "every time", ensure that the logical implications and globally operators correctly capture the temporal dependencies and scope of the conditions, avoiding incorrect nesting or misplacement of operators.

❺ Number of guidelines generated from per failure case. To assess how the number of guidelines generated from a single

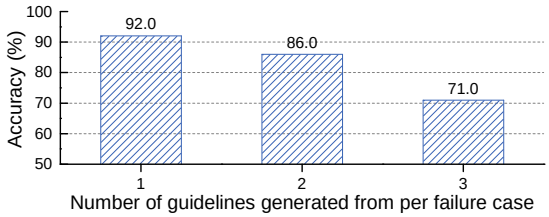


Figure 7: Analysis on the number of guidelines generated from per failure case.

Table 5: Necessity of pattern-specific guidelines

Source \ Target	SynthNL	SpecNL	ConfoNL	LangNL
SynthNL	92.0%	50.0%	60.0%	50.0%

Table 6: Evaluating the generality of induced guidelines

Source \ Target	DeepSeek	Qwen	Gemini
DeepSeek	92.0%	84.0%	84.0%
Qwen	92.0%	85.0%	95.0%
Gemini	95.0%	73.0%	95.0%

failure case affects performance, we evaluate three configurations: generating 1, 2, or 3 guidelines per failure case.

As shown in Figure 7, generating a single guideline yields the highest translation accuracy at 92%. Increasing the number to two reduces accuracy to 86%, and further to 71% with three guidelines. These results indicate that producing one targeted guideline per sample facilitates more precise and reliable learning. In contrast, generating multiple guidelines from the same sample introduces redundancy, which can weaken the effectiveness of downstream translation. Based on this finding, we adopt the one-guideline-per-sample configuration as the default.

Discussion of Findings for RQ4. Analyzing all six experiments, we find that AdaRule performs best when each failure case generates a single, precise guideline. This indicates that targeted, high-quality feedback is more effective than large-scale, undifferentiated data. Based on these results, we recommend a one-to-one, failure-driven learning mode, where each failed sample induces a dedicated guideline to maximize learning efficiency and translation accuracy.

5 Discussion

5.1 Necessity of Pattern-Specific Guidelines

To assess the necessity of using guidelines, we investigate across different datasets to show the linguistic gap between them. We test whether guidelines learned from one dataset (SynthNL) can generalize to other datasets (SpecNL, ConfoNL, and LangNL) using DeepSeek-V3. As shown in Table 5, performance drops from 92.0% on SynthNL to 50.0% on SpecNL and LangNL, and 60.0% on ConfoNL. The results indicate that different datasets exhibit distinct linguistic styles and conventions, making direct guideline transfer across datasets ineffective. This demonstrates the necessity of learning dataset-specific guidelines to capture pattern-specific translation conventions. A limitation of AdaRule is its dependence on the linguistic patterns observed during training. If a completely unseen pattern occurs at test time, the framework may lack a corresponding guideline, resulting in an incorrect translation.

5.2 Generalization of Induced Guidelines

A critical question in assessing our induced guidelines is whether they truly capture dataset-specific transformation patterns or if their effectiveness depends on the LLM used to generate them. To investigate, we fix the dataset and vary the LLM, evaluating whether guidelines from one model remain effective when applied to other models. Using SynthNL dataset, we apply the same set of guidelines, originally generated by one LLM, to translation tasks performed by other LLMs. As shown in Table 6, results remain consistently high across all cross-model configurations. For example, guidelines generated by Gemini-2.5-Flash achieve 95.0% accuracy when applied to DeepSeek-V3, whereas guidelines from Qwen2.5-72B-Instruct yield 95.0% on Gemini and 92.0% on DeepSeek. These results suggest that once pattern-specific knowledge is distilled into guidelines, different LLMs can readily interpret and apply them, demonstrating the strong model-agnostic utility of our guideline representations.

5.3 Data Leakage

For LLM-based methods, a common concern is that the effectiveness may be inflated by potential data leakage. To address this, we evaluated our method in a setting without any induced pattern-specific guidelines, representing a naive LLM-based NL2LTL method without our design. The accuracies on SynthNL, SpecNL, ConfoNL, and LangNL are 36.0%, 72.2%, 51.0%, and 18.0%, which remain much lower than those achieved by our method (92.0%, 83.3%, 83.0%, and 68.0%). This margin demonstrates that the improvements stem from our learning framework rather than data leakage.

5.4 More Naturalistic Requirement Evaluation

To evaluate AdaRule's generalization to more naturalistic specifications, we use 180 NL–LTL pairs from FRET [24], a framework for eliciting and formalizing system requirements. Experimental settings follow our main experiments, with 144 samples for training and 36 for testing. The accuracy improves from 80.6% (using only the basic rules) to 91.7% after applying AdaRule's learned guidelines, confirming its effectiveness in adapting to the linguistic patterns of naturalistic requirements.

5.5 Compare with Rule-Based Method

To investigate potential differences or complementarities between traditional rule-based approaches and the guidelines automatically induced by AdaRule, we conducted an experiment on the SynthNL dataset. We integrated the template-related rules from NL2LTL [21] with the guideline pool generated by AdaRule and tested it on the test set. The accuracy is consistent with the results obtained using only the guideline pool generated by AdaRule. This indicates that the guidelines generated by our method already substantially subsume the capabilities of traditional static templates.

Table 7: Evaluation using Dwyer patterns

Dataset	Absence	Existence	Universality	Precedence	Response
SynthNL	84.4%	100.0%	100.0%	-	95.3%
SpecNL	50.0%	50.0%	87.5%	100.0%	100.0%
ConfoNL	100.0%	80.0%	100.0%	77.8%	75.0%
LangNL	-	87.7%	-	41.9%	-

5.6 Performance across Different Patterns

To provide a granular performance analysis, we manually classify all test sets by Dwyer patterns [17] and evaluate each pattern separately. Table 7 shows generally high accuracy, with near-perfect performance on some patterns (*e.g.*, Universality in SynthNL, Precedence in SpecNL) but lower accuracy on others (*e.g.*, Precedence in LangNL 41.9%, Absence and Existence in SpecNL 50.0%). This result indicates that while the method performs well overall, its performance varies across datasets and patterns.

5.7 Threats to Validity

Internal threats. One internal threat is our method's reliance on the quality and representativeness of training data, as noise or inconsistencies may affect performance. To mitigate this, we use multiple widely adopted NL2LTL datasets capturing diverse linguistic patterns and specification styles, demonstrating generalizability. Another internal threat is potential data leakage, as pretrained LLMs might have encountered some evaluation samples, possibly inflating performance. However, this risk applies equally to all compared methods using the same foundation model. By evaluating under identical settings and strong baselines, we ensure that improvements stem from our method rather than prior exposure.
External threats. All datasets used in our evaluation are in English and focus on specification-related tasks in domains such as software systems and robotics. While covering diverse patterns, results may not generalize to other languages (*e.g.*, Chinese, German) or domains (*e.g.*, medical, legal). Further research is required to assess the method's cross-lingual and cross-domain adaptability.

6 Related Work

6.1 NL2LTL

Early NL2LTL translation efforts include rule-based methods [2, 18, 20, 23, 24, 31, 39, 40, 46] and traditional learning-based models [5, 26, 29, 42, 43, 50]. While rule-based approaches are highly interpretable, they suffer from limited generalization and demand substantial manual engineering. Learning-based approaches, though offering greater flexibility, still face challenges from noisy data, syntactic complexity, and limited robustness [38, 55].

Recent advances leverage large language models (LLMs) to enhance NL2LTL translation, benefiting from their strong language understanding and generalization, with several prompt-driven and modular techniques proposed to better harness them. NL2LTL [21] designs prompts that guide the model in mapping natural language statements to predefined semantic templates. To handle ambiguity, NL2SPEC [13] decomposes the translation process and enables users to iteratively refine erroneous formalizations through interactive sub-translation editing. LfF [56] incorporates user corrections through interactive prompt evolution and employs a retrieval-based dynamic prompt generation mechanism that leverages context similarity to automatically select appropriate prompts. CoT-TL [38] integrates chain-of-thought reasoning, semantic role labeling, and model checking to iteratively refine and validate LTL outputs. Nevertheless, these methods still rely on handcrafted templates or curated examples, limiting their ability to generalize and adapt autonomously to diverse linguistic patterns.

In contrast, our approach automatically learns pattern-adaptive guidelines without manual templates or labeled supervision, using translation failures as implicit signals to help LLMs self-improve and adapt dynamically to diverse linguistic patterns.

6.2 Prompt Engineering

Prompt engineering enhances LLM capabilities by using task-specific instructions without modifying model parameters [47]. A key aspect of prompt engineering is prompt optimization, which iteratively refines input queries to improve the relevance, accuracy, and overall quality of LLM outputs. A prevalent strategy involves ground-truth-based numerical assessments utilizing benchmarks [19, 27, 32, 53]. To conserve samples, some techniques employ LLMs as judges [60] for generating detailed textual feedback [34, 52, 57, 59], offering richer signals with less data. Alternatively, some methods integrate human preferences, either through prescriptive rules or direct feedback [9, 35]. While effective for open-ended tasks, their reliance on extensive human input conflicts with the goal of automation. In our work, we leverage LLMs to automatically learn guidelines from existing failure cases, thereby optimizing prompts.

7 Conclusion

We present AdaRule, a feedback-driven framework that translates NL into LTL without handcrafted templates or human intervention. It integrates an LLM-based Translator and Learner in a self-improving loop, autonomously learning pattern-specific translation rules from failures. Experiments show that AdaRule improves average accuracy by over 21% across four NL2LTL benchmarks and three large language models. In future work, we plan to extend AdaRule to cross-lingual settings for non-English requirements.

Acknowledgments

This research is supported by the Fundamental Research Funds for the Central Universities (ZYGX2024XJ036). The work is supported by the NSFC (Grant Nos. 92582203, 62502077, 62372181). This work is supported by the Shanghai Collaborative Innovation Center of Trusted Industry Internet Software. It is also supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 2, Award No. MOE-T2EP20124-0016. It is also supported by the CAS Project for Young Scientists in Basic Research (Grant No. YSBR-040), ISCAS Basic Research (Grant Nos. ISCAS-JCZD-202406, ISCAS-JCZD-202302), ISCAS New Cultivation Project ISCAS-PYFX-202201. This work was sponsored by the Open Project of Shanghai Driverless Train Control System Engineering Technology Research Center (No. SUTC-2024KT-03).

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

[1] AIBase. 2025. *AIBase Leaderboard – Text Generation Rankings.* https://model.aibase.com/zh/leaderboard/text-generation Accessed: November 2025.

[2] Chetan Arora, Mehrdad Sabetzadeh, Lionel Briand, and Frank Zimmer. 2017. Automated Extraction and Clustering of Requirements Glossary Terms. *IEEE Transactions on Software Engineering* 43, 10 (2017), 918–945. doi:10.1109/TSE.2016.2635134

[3] Marco Autili, Lars Grunske, Markus Lumpe, Patrizio Pelliccione, and Antony Tang. 2015. Aligning qualitative, real-time, and probabilistic property specification patterns using a structured english grammar. *IEEE Transactions on Software Engineering* 41, 7 (2015), 620–638.

[4] Marisol Barrientos, Karolin Winter, and Stefanie Rinderle-Ma. 2024. Automatic Extraction and Formalization of Temporal Requirements from Text: A Survey. In *International Conference on Enterprise Design, Operations, and Computing*. Springer, 259–278.

[5] Matthew Berg, Deniz Bayazit, Rebecca Mathew, Ariel Rotter-Aboyoun, Ellie Pavlick, and Stefanie Tellex. 2020. Grounding language to landmarks in arbitrary outdoor environments. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 208–215.

[6] Elfia Bezou-Vrakatseli, Oana Cocarascu, and Sanjay Modgil. 2025. Can Large Language Models Understand Argument Schemes?. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 13666–13681. doi:10.18653/v1/2025.findings-acl.702

[7] Andrea Brunello, Angelo Montanari, and Mark Reynolds. 2019. Synthesis of LTL formulas from natural language texts: State of the art and research directions. In *26th International symposium on temporal representation and reasoning (TIME 2019)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 17–1.

[8] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[9] Yongchao Chen, Jacob Arkin, Yilun Hao, Yang Zhang, Nicholas Roy, and Chuchu Fan. 2024. PRompt Optimization in Multi-Step Tasks (PROMST): Integrating Human Feedback and Heuristic-based Sampling. In *EMNLP*. Association for Computational Linguistics, 3859–3920.

[10] Yongchao Chen, Rujul Gandhi, Yang Zhang, and Chuchu Fan. 2023. Nl2tl: Transforming natural languages to temporal logics using large language models. *arXiv preprint arXiv:2305.07766* (2023).

[11] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, et al. 2022. Scaling Instruction-Finetuned Language Models. arXiv:2210.11416 [cs.LG] https://arxiv.org/abs/2210.11416

[12] Edmund M. Clarke and Jeannette M. Wing. 1996. Formal methods: state of the art and future directions. *ACM Comput. Surv.* 28, 4 (Dec. 1996), 626–643. doi:10.1145/242223.242257

[13] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. 2023. nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models. arXiv:2303.04864 [cs.LO] https://arxiv.org/abs/2303.04864

[14] cRick. 2023. NL2LTL-Synthetic-Dataset. https://huggingface.co/datasets/cRick/NL-to-LTL-Synthetic-Dataset

[15] Alexandre Duret-Lutz. 2022. Spot's temporal logic formulas. *Manual for Spot* 2, 3 (2022).

[16] Alexandre Duret-Lutz, Etienne Renault, Maximilien Colange, Florian Renkin, Alexandre Gbaguidi Aisse, Philipp Schlehuber-Caissier, Thomas Medioni, Antoine Martin, Jérôme Dubois, Clément Gillard, and Henrich Lauko. 2022. From Spot 2.0 to Spot 2.10: What's New?. In *CAV (Lecture Notes in Computer Science, Vol. 13372)*. Springer, 174–187. doi:10.1007/978-3-031-13188-2_9

[17] M.B. Dwyer, G.S. Avrunin, and J.C. Corbett. 1999. Patterns in property specifications for finite-state verification. In *Proceedings of the 1999 International Conference on Software Engineering (IEEE Cat. No.99CB37002)*. 411–420. doi:10.1145/302405.302672

[18] M.B. Dwyer, G.S. Avrunin, and J.C. Corbett. 1999. Patterns in property specifications for finite-state verification. In *Proceedings of the 1999 International Conference on Software Engineering (IEEE Cat. No.99CB37002)*. 411–420. doi:10.1145/302405.302672

[19] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. In *ICML*. OpenReview.net.

[20] Cameron Finucane, Gangyuan Jing, and Hadas Kress-Gazit. 2010. LTLMoP: Experimenting with language, Temporal Logic and robot control. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 1988–1993. doi:10.1109/IROS.2010.5650371

[21] Francesco Fuggitti and Tathagata Chakraborti. 2023. Nl2ltl–a python package for converting natural language (nl) instructions to linear temporal logic (ltl) formulas. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 16428–16430.

[22] Samuele Germiniani, Daniele Nicoletti, and Graziano Pravadelli. 2025. A Systematic Literature Review on Mining LTL Specifications. *IEEE Access* (2025).

[23] Shalini Ghosh, Daniel Elenius, Wenchao Li, Patrick Lincoln, Natarajan Shankar, and Wilfried Steiner. 2016. ARSENAL: Automatic Requirements Specification Extraction from Natural Language. arXiv:1403.3142 [cs.CL] https://arxiv.org/abs/1403.3142

[24] Dimitra Giannakopoulou, Anastasia Mavridou, Julian Rhein, Thomas Pressburger, Johann Schumann, and Nija Shi. 2020. Formal requirements elicitation with FRET. In *International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ-2020)*.

[25] Luis Gomes, João Paulo Barros, Anikó Costa, Rui Pais, and Filipe Moutinho. 2005. Towards usage of formal methods within embedded systems co-design. In *2005 IEEE Conference on Emerging Technologies and Factory Automation*, Vol. 1. IEEE, 4–pp.

[26] Nakul Gopalan, Dilip Arumugam, Lawson LS Wong, and Stefanie Tellex. 2018. Sequence-to-Sequence Language Grounding of Non-Markovian Task Specifications.. In *Robotics: Science and Systems*, Vol. 2018.

[27] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers. In *ICLR*. OpenReview.net.

[28] Osman Hasan and Sofiene Tahar. 2015. Formal verification methods. In *Encyclopedia of Information Science and Technology, Third Edition*. IGI Global Scientific Publishing, 7162–7170.

[29] Jie He, Ezio Bartocci, Dejan Ničković, Haris Isakovic, and Radu Grosu. 2022. DeepSTL – From English Requirements to Signal Temporal Logic. arXiv:2109.10294 [cs.CL] https://arxiv.org/abs/2109.10294

[30] Eric Hsiung, Hiloni Mehta, Junchi Chu, Xinyu Liu, Roma Patel, Stefanie Tellex, and George Konidaris. 2022. Generalizing to new domains by mapping natural language to lifted LTL. In *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 3624–3630.

[31] Eric Hsiung, Hiloni Mehta, Junchi Chu, Xinyu Liu, Roma Patel, Stefanie Tellex, and George Konidaris. 2022. Generalizing to New Domains by Mapping Natural Language to Lifted LTL. In *2022 International Conference on Robotics and Automation (ICRA)*. 3624–3630. doi:10.1109/ICRA46639.2022.9812169

[32] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *CoRR* abs/2310.03714 (2023).

[33] Jianwen Li, Yinbo Yao, Geguang Pu, Lijun Zhang, and Jifeng He. 2014. Aalta: an LTL satisfiability checker over Infinite/Finite traces. 731–734. doi:10.1145/2635868.2661669

[34] Yafu Li, Xuyang Hu, Xiaoye Qu, Linjie Li, and Yu Cheng. 2025. Test-Time Preference Optimization: On-the-Fly Alignment via Iterative Textual Feedback. *CoRR* abs/2501.12895 (2025).

[35] Xiaoqiang Lin, Zhongxiang Dai, Arun Verma, See-Kiong Ng, Patrick Jaillet, and Bryan Kian Hsiang Low. 2024. Prompt Optimization with Human Feedback. *CoRR* abs/2405.17346 (2024).

[36] Jason Xinyu Liu, Ankit Shah, George Konidaris, Stefanie Tellex, and David Paulius. 2024. Lang2LTL-2: Grounding Spatiotemporal Navigation Commands Using Large Language and Vision-Language Models. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.

[37] Jason Xinyu Liu, Ziyi Yang, Ifrah Idrees, Sam Liang, Benjamin Schornstein, Stefanie Tellex, and Ankit Shah. 2023. Lang2ltl: Translating natural language commands to temporal robot task specification. *arXiv preprint arXiv:2302.11649* (2023).

[38] Kumar Manas, Stefan Zwicklbauer, and Adrian Paschke. 2024. CoT-TL: Low-Resource Temporal Knowledge Representation of Planning Instructions Using Chain-of-Thought Reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 9636–9643. doi:10.1109/iros58592.2024.10801817

[39] Regan Meloche, Daniel Amyot, and John Mylopoulos. 2023. Towards Legal Contract Formalization with Controlled Natural Language Templates. In *2023 IEEE 31st International Requirements Engineering Conference (RE)*. 317–322. doi:10.1109/RE57278.2023.00042

[40] Claudio Menghi, Christos Tsigkanos, Mehrnoosh Askarpour, Patrizio Pelliccione, Gricel Vázquez, Radu Calinescu, and Sergio García. 2023. Mission Specification Patterns for Mobile Robots: Providing Support for Quantitative Properties. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2741–2760. doi:10.1109/TSE.2022.3230059

[41] Allen P Nikora and Galen Balcom. 2009. Automated identification of LTL patterns in natural language requirements. In *2009 20th International Symposium on Software Reliability Engineering*. IEEE, 185–194.

[42] Yoonseon Oh, Roma Patel, Thao Nguyen, Baichuan Huang, Ellie Pavlick, and Stefanie Tellex. 2019. Planning with state abstractions for non-markovian task specifications. *arXiv preprint arXiv:1905.12096* (2019).

[43] Roma Patel, Ellie Pavlick, and Stefanie Tellex. 2020. Grounding Language to Non-Markovian Tasks with No Supervision of Task Specifications.. In *Robotics: Science and Systems*, Vol. 2020.

[44] Amir Pnueli. 1977. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*. 46–57. doi:10.1109/SFCS.1977.32

[45] Markus Roggenbach, Antonio Cerone, Bernd-Holger Schlingloff, Gerardo Schneider, and Siraj Ahmed Shaikh. 2021. Formal Methods for Software Engineering. *Springer Nature Switzerland, Gewerbestrasse* 11 (2021), 6330.

[46] Driss Sadoun, Catherine Dubois, Yacine Ghamri-Doudane, and Brigitte Grau. 2013. From Natural Language Requirements to Formal Specification Using an Ontology. In *2013 IEEE 25th International Conference on Tools with Artificial Intelligence*. 755–760. doi:10.1109/ICTAI.2013.116

[47] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927* (2024).

[48] Marco Sgroi, Luciano Lavagno, and Alberto Sangiovanni-Vincentelli. 2002. Formal models for embedded system design. *IEEE Design & Test of Computers* 17, 2 (2002), 14–27.

[49] TIMETOACT Group. 2025. *LLM Benchmarks – April 2025*. https://www.timetoact-group.at/en/insights/llm-benchmarks/llm-benchmarks-april-2025 Accessed: November 2025.

[50] Christopher Wang, Candace Ross, Yen-Ling Kuo, Boris Katz, and Andrei Barbu. 2021. Learning a natural-language to LTL executable semantic parser for grounded robotics. In *Conference on Robot Learning*. PMLR, 1706–1718.

[51] Jun Wang, David Smith Sundarsingh, Jyotirmoy V. Deshmukh, and Yiannis Kantaros. 2025. ConformalNL2LTL: Translating Natural Language Instructions into Temporal Logic Formulas with Conformal Correctness Guarantees. arXiv:2504.21022 [cs.CL] https://arxiv.org/abs/2504.21022

[52] Wenyi Wang, Hisham A. Alyahya, Dylan R. Ashley, Oleg Serikov, Dmitrii Khizbullin, Francesco Faccio, and Jürgen Schmidhuber. 2024. How to Correctly do Semantic Backpropagation on Language-based Agentic Systems. *CoRR* abs/2412.03624 (2024).

[53] Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P. Xing, and Zhiting Hu. 2024. PromptAgent: Strategic Planning with Language Models Enables Expert-level Prompt Optimization. In *ICLR*. OpenReview.net.

[54] Jim Woodcock, Peter Gorm Larsen, Juan Bicarregui, and John Fitzgerald. 2009. Formal methods: Practice and experience. *ACM computing surveys (CSUR)* 41, 4 (2009), 1–36.

[55] Yi Wu, Zikang Xiong, Yiran Hu, Shreyash S. Iyengar, Nan Jiang, Aniket Bera, Lin Tan, and Suresh Jagannathan. 2025. SELP: Generating Safe and Efficient Task Plans for Robot Agents with Large Language Models. arXiv:2409.19471 [cs.RO] https://arxiv.org/abs/2409.19471

[56] Yilongfei Xu, Jincao Feng, and Weikai Miao. 2024. Learning from Failures: Translation of Natural Language Requirements into Linear Temporal Logic with Large Language Models. In *2024 IEEE 24th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 204–215.

[57] Cilin Yan, Jingyun Wang, Lin Zhang, Ruihui Zhao, Xiaopu Wu, Kai Xiong, Qingsong Liu, Guoliang Kang, and Yangyang Kang. 2024. Efficient and Accurate Prompt Optimization: the Benefit of Memory in Exemplar-Guided Reflection. *CoRR* abs/2411.07446 (2024).

[58] Zonglin Yang, Li Dong, Xinya Du, Hao Cheng, Erik Cambria, Xiaodong Liu, Jianfeng Gao, and Furu Wei. 2024. Language Models as Inductive Reasoners. arXiv:2212.10923 [cs.CL] https://arxiv.org/abs/2212.10923

[59] Mert Yüksekgönül, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. *CoRR* abs/2406.07496 (2024).

[60] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS*.